

Rapport Individuel - SAÉ 2.01

Timothy Maume

1. Introduction

Dans le cadre du projet Briqu'IUTO, dont l'objectif était de développer une application Java de gestion de collection de LEGO, je me suis concentré sur la conception et l'implémentation du cœur de l'application. Mon travail a consisté à mettre en place la connexion à la base de données, à concevoir les classes liés aux boîtes de LEGO, et à développer la logique de service pour respecter les couches de notre architecture.

2. Mes contributions techniques

Mon travail s'est réparti sur les trois parties du logiciel:

A. La couche d'accès aux données (BD / JDBC) J'ai eu la responsabilité d'établir le lien entre notre code Java et la base de données relationnelle via JDBC.

- **ConnexionMySQL** : J'ai implémenté cette classe pour gérer l'ouverture, la fermeture et le maintien de la connexion au serveur MariaDB de manière sécurisée.
- **Les classes d'accès (BoiteBD, Contenu, SupplementBD)** : J'ai rédigé les requêtes SQL (SELECT, INSERT, UPDATE, JOIN) permettant de manipuler les données. Un de mes objectifs principaux a été de faire en sorte que ces classes renvoient directement des objets métiers Java propres, en évitant les structures de données intermédiaires inutiles.

B. La couche de logique métier (Services) Pour éviter d'avoir des classes "fourre-tout", j'ai structuré la logique autour de services spécialisés isolant les traitements complexes de l'interface utilisateur.

- **BoiteService** : C'est le chef d'orchestre de la gestion des boîtes. J'y ai implémenté l'algorithme d'assemblage (**chargerBoiteComplete**) qui rassemble les pièces, les figurines et les sous-boîtes. J'ai également géré la logique de la méthode **composerBoitePersonnalisee()** demandée par le client.
- **CollectionService** : J'ai développé ce service pour permettre à l'utilisateur de gérer sa collection personnelle, d'indiquer l'état de ses boîtes (complètes ou incomplètes) et de calculer automatiquement la liste des pièces manquantes.

C. Les modèles de données et la gestion des erreurs

- **Les entités (Boite, BoiteQuantite, BoiteStats, CollectionItem)** : J'ai modélisé ces classes en respectant strictement l'encapsulation. Elles contiennent les propriétés fondamentales et des méthodes utiles (comme le calcul du nombre total de pièces).
- **BoiteIdentiqueException** : Pour respecter l'architecture logicielle, j'ai créé cette exception personnalisée. Lorsqu'un utilisateur tente de créer une boîte personnalisée avec des pièces identiques à une boîte existante, le **BoiteService** lève cette erreur au lieu d'interagir directement avec la console. Cela permet à l'interface de l'attraper et de demander confirmation à l'utilisateur proprement.

3. Difficultés rencontrées et solutions apportées

Le défi majeur de ma partie a été d'éviter le code "spaghetti". Au début du développement, la logique métier, l'accès aux données et les interactions utilisateur avaient tendance à s'entremêler. J'ai dû mener un important travail de refactoring pour séparer clairement les responsabilités. Par exemple, j'ai dû concevoir le **BoiteService** de manière à ce qu'il interroge **BoiteBD**, **Contenu** et **SupplementBD** pour assembler un objet **Boite** complet en mémoire, sans jamais faire appel à des éléments d'interface (**Scanner** ou **System.out.println**).

4. Conclusion et apprentissages

Ce projet m'a permis de consolider mes compétences en programmation orientée objet en Java. J'ai particulièrement apprécié le travail sur l'architecture logicielle : comprendre qu'une application robuste nécessite une séparation stricte entre la base de données, la logique métier et l'interface. La maîtrise de l'API JDBC et la gestion propre des exceptions constituent pour moi les acquis les plus importants de cette SAE, car ce sont des méthodes de travail directement transposables dans le monde professionnel.